

EXPERIMENT REPORT

A Comparative Study of Constructive Heuristics for the Orienteering Problem

João Pereira [Universidade Federal do Rio Grande do Norte | joao.pereira.016@edu.ufrn.br]

✉ Departamento de Informática e Matemática Aplicada

Universidade Federal do Rio Grande do Norte

Campus Universitário – Lagoa Nova, Natal – RN, 59072-970, Brasil

Abstract. This report studies constructive insertion heuristics for the Orienteering Problem (OP), an NP-hard routing problem that maximizes collected reward under a travel-budget constraint. Five deterministic selection criteria and three guided semi-randomized mechanisms are evaluated on 89 benchmark instances from the Tsiligirides and Chao sets. In the elementary-criterion experiment, each criterion–instance combination was executed 5,000 times. Solution quality is summarized by the median collected score, and route structure is compared through one arc-based Jaccard distance, from which global and local diversity measures are derived. Total-envy obtained the largest mean deterministic improvement over marginal-smart (approximately 4.07%). The best semi-randomized method obtained approximately 3.80% while retaining about 69–71% of the global diversity of pure-random insertion. These results indicate that guided randomization can increase structural exploration without the severe quality loss produced by unrestricted random selection.

Keywords: Orienteering Problem, constructive heuristic, insertion heuristic, guided randomization, structural diversity

1 Introduction

The Orienteering Problem (OP) combines vertex selection and route construction: only a subset of rewarding vertices can be visited because the route must fit a limited travel budget. It appears in tourist itinerary design, surveillance, mobile data collection, and other settings in which the value of a route depends on what is visited as well as on how it is traversed [Vansteenwegen *et al.*, 2011; Gunawan *et al.*, 2016].

This report investigates a generalized constructive insertion heuristic. The candidate-generation and feasibility mechanisms are fixed, while a selection criterion determines which feasible vertex-position pair is inserted next. This separation supports a controlled comparison of score-only, cost-only, score–cost, total-route, and semi-randomized selection policies.

The contributions are threefold: (i) a common formulation for the evaluated constructive criteria; (ii) an experimental comparison on 90 classical instances; and (iii) a joint analysis of objective value and structural route diversity. Section 2 defines the problem and its complexity, Section 3 summarizes related work, Section 4 presents the implemented heuristics, Section 5 reports the experiments, and Section 6 concludes the report.

2 Problem Description and Complexity

Let $G = (V, A)$ be a directed graph with designated start and finish vertices $s, t \in V$. Every optional vertex $i \in V \setminus \{s, t\}$ has a nonnegative score p_i , every arc $(i, j) \in A$ has travel cost c_{ij} , and B is the available travel budget. A feasible solution is an elementary path

$$R = (v_0 = s, v_1, \dots, v_k, v_{k+1} = t) \quad (1)$$

whose travel cost and collected score are, respectively,

$$C(R) = \sum_{r=0}^k c_{v_r v_{r+1}} \leq B, \quad S(R) = \sum_{r=1}^k p_{v_r}. \quad (2)$$

The objective is to find a feasible route R^* such that

$$R^* \in \arg \max_{R: C(R) \leq B} S(R). \quad (3)$$

The OP is NP-hard; it combines aspects of the traveling-salesperson and knapsack problems, so exact solution becomes computationally expensive as the instance grows [Golden *et al.*, 1987; Vansteenwegen *et al.*, 2011]. This complexity motivates constructive heuristics that quickly produce feasible solutions without a guarantee of optimality.

3 Related Work

Tsiligirides [Tsiligirides, 1984] introduced influential stochastic and deterministic heuristics together with benchmark instances that remain widely used. Golden, Levy, and Vohra [Golden *et al.*, 1987] formalized the OP and discussed its relation to established routing problems. Chao, Golden, and Wail [Chao *et al.*, 1996] later proposed a fast heuristic and additional benchmark instances.

Insertion rules are a recurring component of OP algorithms. Kantor and Rosenwein [Kantor and Rosenwein, 1992] used reward relative to incremental insertion time for the OP with time windows. Related best-insertion mechanisms also appear in algorithms for the Team Orienteering Problem; Dang *et al.* [Dang *et al.*, 2013], for example, incorporate a best-insertion procedure into a PSO-inspired method.

The literature subsequently expanded toward exact methods, local search, tabu search, variable-neighborhood methods, evolutionary algorithms, and hybrid approaches; surveys by Vansteenwegen *et al.* [Vansteenwegen *et al.*, 2011] and Gunawan *et al.* [Gunawan *et al.*, 2016] provide broader

taxonomies. More recent work has continued on both fronts: Kobeaga et al. [Kobeaga *et al.*, 2018] developed an evolutionary algorithm for the classical OP, while Kobeaga et al. [Kobeaga *et al.*, 2024] revisited branch-and-cut for large-scale instances. These methods are substantially more elaborate than a one-pass construction. The present work therefore does not claim a new state-of-the-art OP solver. Its scope is narrower: it isolates the effect of the insertion-selection rule within one constructive framework and establishes the baseline required for the first course project.

4 Implemented Constructive Heuristics

4.1 Common Insertion Framework

Let R_t be the route at iteration t , initially $R_0 = (s, t)$, and let U_t contain the unvisited optional vertices. Inserting $u \in U_t$ between consecutive route vertices i and j produces $R_t \oplus_{(i,j)} u$. The marginal score and insertion cost are defined separately as

$$\begin{aligned} \Delta S(u) &= p_u, \\ \Delta C(i, u, j) &= c_{iu} + c_{uj} - c_{ij}. \end{aligned} \quad (4)$$

Let $A(R_t)$ denote the consecutive arcs traversed by R_t . A triple (u, i, j) belongs to the candidate set $\mathcal{C}(R_t)$ if and only if all four conditions below hold:

1. $u \in U_t$;
2. $(i, j) \in A(R_t)$;
3. $(i, u) \in A$ and $(u, j) \in A$; and
4. $C(R_t) + \Delta C(i, u, j) \leq B$.

At each iteration, the algorithm (1) enumerates $\mathcal{C}(R_t)$, (2) applies one selection rule, (3) permanently inserts the selected vertex, and (4) stops when the candidate set is empty. If the current route contains r optional vertices and u optional vertices remain, the common framework examines at most $u(r + 1)$ vertex–position pairs in that iteration. Runtime was not an evaluation objective and is not compared among criteria. In particular, the present prototype reconstructs and reevaluates complete candidate routes; therefore, timing it would mainly measure an unoptimized shared implementation rather than the intrinsic merit of a selection rule. Incremental score and insertion-cost updates are left as an implementation improvement.

4.2 Deterministic Selection Criteria

Table 1 defines the five elementary criteria in their marginal form. Maximization is used except for lazy, which minimizes insertion cost. Greedy and lazy have the same ordering in their total and marginal forms during one construction iteration. The nonlinear criteria do not, so their exact variant must always be named.

Table 1. Elementary deterministic selection criteria.

Name	Direction	Candidate value
greedy	maximize	ΔS
lazy	minimize	ΔC
smart	maximize	$\Delta S/\Delta C$
wise	maximize	$\Delta S/(\Delta C)^2$
envy	maximize	$(\Delta S)^2/\Delta C$

For greedy and lazy, ranking candidates by total-route values $S(R')$ and $C(R')$ is equivalent to ranking them by ΔS and ΔC within one construction iteration, because the current route is fixed. This equivalence does not hold for nonlinear score–cost ratios.

When travel costs satisfy the triangle inequality, $\Delta C \geq 0$. In the implementation, a denominator no greater than $\text{f64}::\text{EPSILON}$ yields $+\infty$ when the corresponding score is positive and zero otherwise. This same guard is used by all marginal and total ratio criteria.

The implementation confirms that the elementary plots in Figures 1–4 use marginal-smart, marginal-wise, and marginal-envy. This resolves the inconsistent early formulas in the original manuscript, which mixed marginal and total-route notation.

4.3 Marginal and Total Variants

Let R' be the candidate route after an insertion. The total-route variants rank candidates using $S(R')$ and $C(R')$, as shown in Table 2. The two formulations need not induce the same ordering because the ratios are nonlinear.

Table 2. Marginal and total formulations of the score–cost criteria.

Criterion	Marginal	Total
smart	$\Delta S/\Delta C$	$S(R')/C(R')$
wise	$\Delta S/(\Delta C)^2$	$S(R')/C(R')^2$
envy	$(\Delta S)^2/\Delta C$	$S(R')^2/C(R')$

4.4 Tie Handling and Reproducibility

In the evaluated implementation, unvisited vertices are stored in Rust’s HashSet. A deterministic selector retains the first enumerated candidate when its comparator reports equality. Because hash iteration order is not a semantic ordering and can change between independently initialized sets, repeated executions may follow different construction trajectories. Selectors whose names end in `random_tie` instead use reservoir-style tie handling, which selects uniformly among all candidates tied for best.

The first behavior is implementation-dependent nondeterminism, not an explicitly randomized heuristic.

4.5 Guided Semi-Randomized Selection

Three mechanisms introduce explicit randomization while preserving the same feasibility test:

- `selected_random` samples one retained deterministic criterion at each iteration and uses it to rank all feasible candidates;
- `selected_weighted_random` samples the criterion from a nonuniform distribution based on deterministic performance; and
- `selected_random_random_tie` also resolves tied best candidates randomly.

The comparison also includes `random_feasible_insertion`, which samples one candidate uniformly from $\mathcal{C}(R_t)$. The selected pool is {marginal-envy, total-envy, marginal-wise}. The weighted selector uses raw roulette weights 0.963, 0.971, and 0.948, respectively; division by their sum gives approximate selection probabilities of 33.41%, 33.69%, and 32.89%.

5 Computational Experiments

5.1 Methodology and Reproducibility

Each method–instance experiment executed through `single_results` uses 5,000 independent constructions. Objective-value summaries use all 5,000 scores. Structural metrics use reservoir sampling to retain 1,000 execution routes uniformly without replacement from those runs; structurally identical routes remain repeated observations. The same function is used for the semi-randomized battery and for explicitly supplied deterministic selectors.

The implementation is written in Rust.

- hardware: AMD Ryzen 7 5700X (16) @ 4.67 GHz, 31.27 GiB .
- software: NixOS 26.05 (Yarara) x86_64, Rust 1.97.1.

The current results are descriptive. Error bars represent standard errors across benchmark instances and are not significance tests. No inferential statistical test was performed, so the reported differences should not be interpreted as evidence of statistical significance. A future confirmatory comparison could use instances as paired blocks and should report effect sizes in addition to corrected p -values.

5.2 Benchmark Instances

As inventoried in the supplied experiment description, the benchmark comprises 89 classical instances. The Tsiligrades collection contributes 18 instances with 32 vertices, 11 with 21 vertices, and 20 with 33 vertices [Tsiligrades, 1984]. The Chao collection contributes 14 instances with 64 vertices and 26 with 66 vertices [Chao *et al.*, 1996]. The original instance sets are distributed by KU Leuven [KU Leuven, Mechatronic Systems, Robotics and Biomechanics, 2026], and the same local instance files should be used for every method.

5.3 Evaluation Measures

For method m and instance p , let $M_{m,p}$ be the median score over the recorded runs for that method–instance pair. Two normalizations are used, and each figure names its reference explicitly. Relative-to-best quality is

$$q_{m,p} = \frac{M_{m,p}}{\max_{m' \in \mathcal{M}} M_{m',p}}, \quad Q_m = \frac{1}{|P|} \sum_{p \in P} q_{m,p}. \quad (5)$$

For comparisons with one reference method b , the percentage difference is

$$\delta_{m,p}^Q = 100 \frac{M_{m,p} - M_{b,p}}{M_{b,p}}. \quad (6)$$

Structural diversity is defined once for the entire report. Represent route R by the set $A(R)$ of directed traversed arcs,

including arcs incident to the start and finish. The Jaccard arc distance is

$$d(R_i, R_j) = 1 - \frac{|A(R_i) \cap A(R_j)|}{|A(R_i) \cup A(R_j)|}. \quad (7)$$

For an indexed route sample $\mathcal{R} = (R_1, \dots, R_n)$, in which repeated route structures remain repeated observations, global diversity is the average distance over all unordered pairs,

$$D_{\text{global}}(\mathcal{R}) = \frac{2}{n(n-1)} \sum_{1 \leq i < j \leq n} d(R_i, R_j). \quad (8)$$

Let $N_k(R_i)$ contain the $k = 5$ nearest routes to R_i under the same distance. Local diversity is

$$D_{\text{local}}(\mathcal{R}) = \frac{1}{n} \sum_{i=1}^n \left(\frac{1}{k} \sum_{R_j \in N_k(R_i)} d(R_i, R_j) \right). \quad (9)$$

Adjacency-based distances and nearest-neighbor diversity contributions are used in routing metaheuristics; the present metric follows that structural principle but uses a normalized Jaccard distance over directed arcs [Vidal *et al.*, 2013].

For a diversity reference b , the plotted percentage difference is defined when $D_{b,p} > 0$:

$$\delta_{m,p}^D = 100 \frac{D_{m,p} - D_{b,p}}{D_{b,p}}. \quad (10)$$

Equations (6) and (10) include the factor 100 because the corresponding axes and reported values are percentages.

5.4 Elementary Criteria

The numerical values discussed below were transcribed from the supplied manuscript and plots; raw result tables were not supplied for independent recalculation.

Figure 1 reports the median score for every instance. The landscape orientation gives the dense instance labels more horizontal space than the original one-column placement.

Because absolute score scales differ by instance, Figure 2 uses Equation (5). Envy achieves the highest aggregate value, followed by wise and smart; greedy and lazy perform substantially worse. This is consistent with greedy and lazy considering only one of the two relevant dimensions.

Figures 3 and 4 use the single diversity definition in Section 5.3. Smart and envy have similar global diversity (about 0.29), whereas wise is lower (about 0.24). Locally, wise is the most diverse; smart and especially envy form groups of nearly identical routes. Thus, global spread and local redundancy describe different properties and must not be conflated.

5.5 Marginal versus Total Criteria

Marginal-smart is the common reference in this comparison. The mean of Equation (6) is approximately 2.04% for total-smart, 2.89% for marginal-envy, and 4.07% for total-envy. Marginal-wise is approximately 0.72% above the reference,

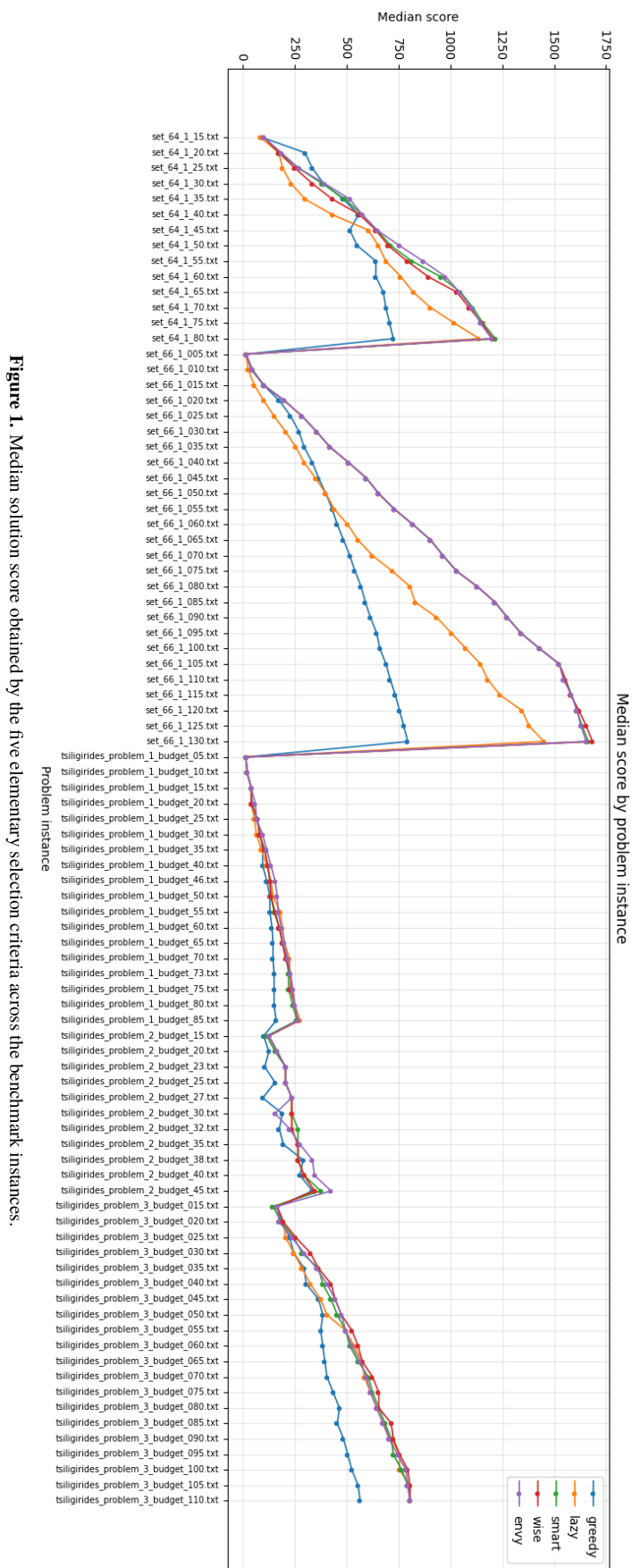


Figure 1. Median solution score obtained by the five elementary selection criteria across the benchmark instances.

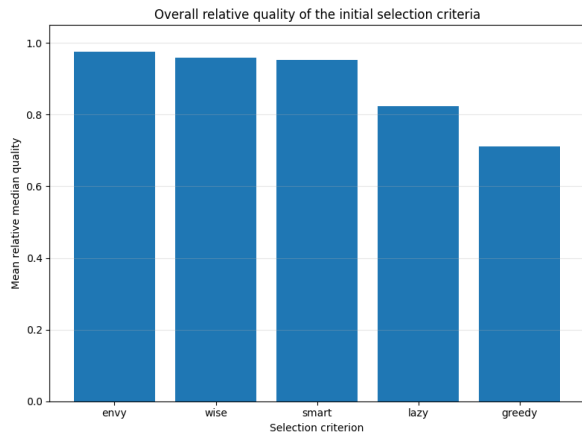


Figure 2. Mean relative median quality of the five elementary criteria. For each instance, the best median among these criteria is normalized to 1.

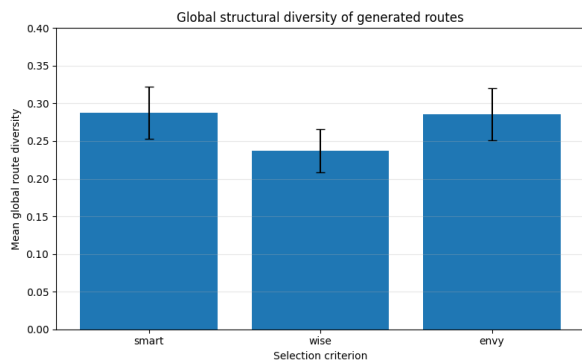


Figure 3. Mean global route diversity for smart, wise, and envy. Error bars represent standard errors across benchmark instances.

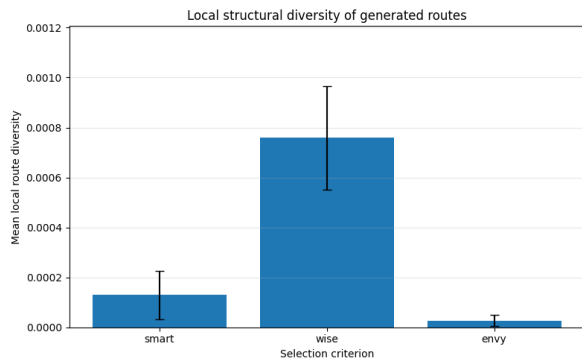


Figure 4. Mean local route diversity for smart, wise, and envy, using the five nearest routes. Error bars represent standard errors across benchmark instances.

whereas total-wise is approximately 0.41% below it. Total-route information is therefore beneficial for some nonlinear criteria but not universally.

The three strongest deterministic alternatives are total-envy, marginal-envy, and total-smart. Figure 6 compares them directly with marginal-smart.

Their diversity need not improve together with score. Figure 7 shows percentage differences relative to marginal-smart using Equation (10).

5.6 Semi-Randomized Criteria

The comparison contains marginal-smart, total-smart, marginal-envy, total-envy, the three guided semi-randomized methods, and pure-random insertion. Figure 8 shows the medians by instance; the landscape orientation reduces label

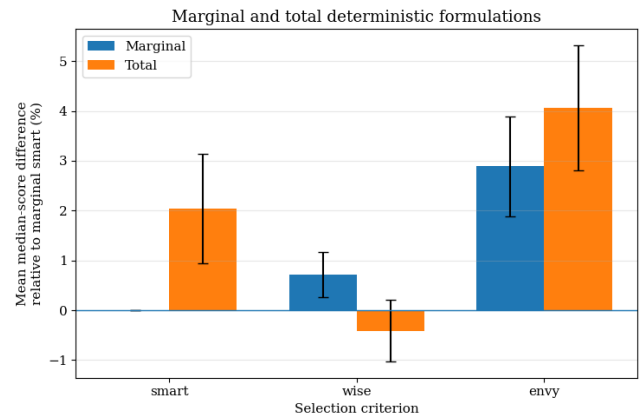


Figure 5. Mean median-score difference between marginal and total criteria, relative to marginal-smart. Error bars are standard errors across benchmark instances.

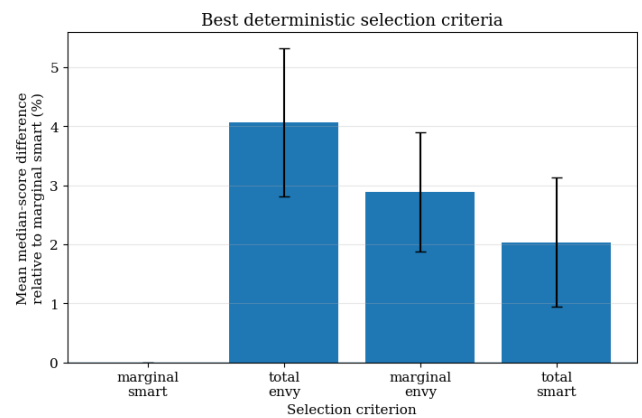


Figure 6. Mean median-score difference of the strongest deterministic criteria relative to marginal-smart.

compression.

Selected-weighted-random, selected-random, and selected-random-random-tie improve mean median score over marginal-smart by approximately 3.80%, 3.70%, and 3.55%, respectively. Pure random is approximately 44.17% below marginal-smart, showing that randomization without guidance is not competitive.

For diversity, pure random is the reference. The strongest deterministic methods are about 69–72% below it globally, while the semi-randomized methods are about 29–31% below it. The latter therefore retain roughly 69–71% of pure-random global diversity while remaining close to the best deterministic quality. Local-diversity percentage differences are less stable because the absolute denominators can be near zero and should be interpreted cautiously.

The method names are rasterized too densely in the three panels below and in the by-instance plot.

Figure 10 summarizes the trade-off.

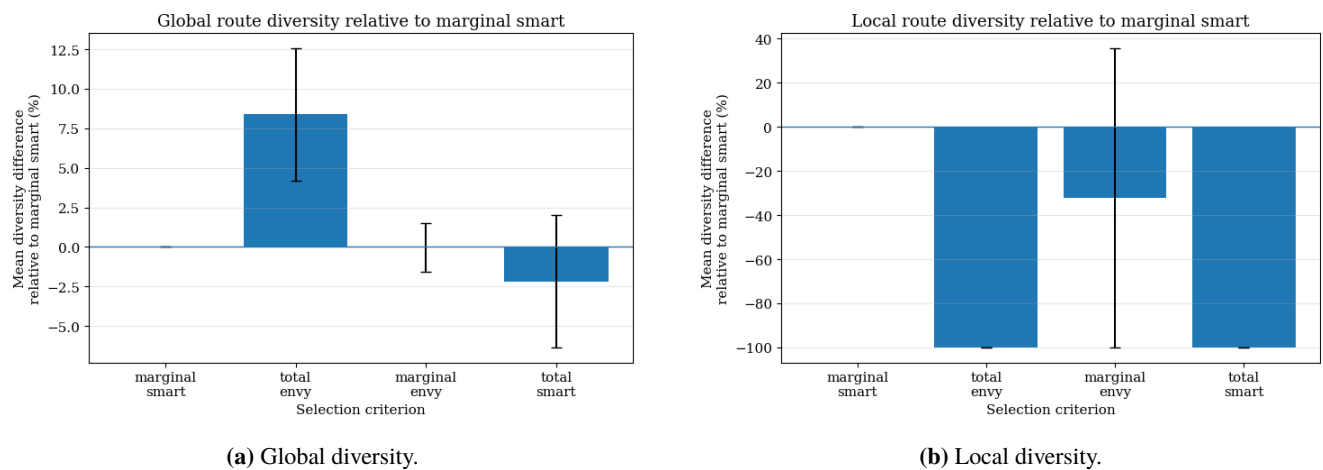


Figure 7. Structural diversity of selected deterministic criteria relative to marginal-smart. Error bars are standard errors across instances.

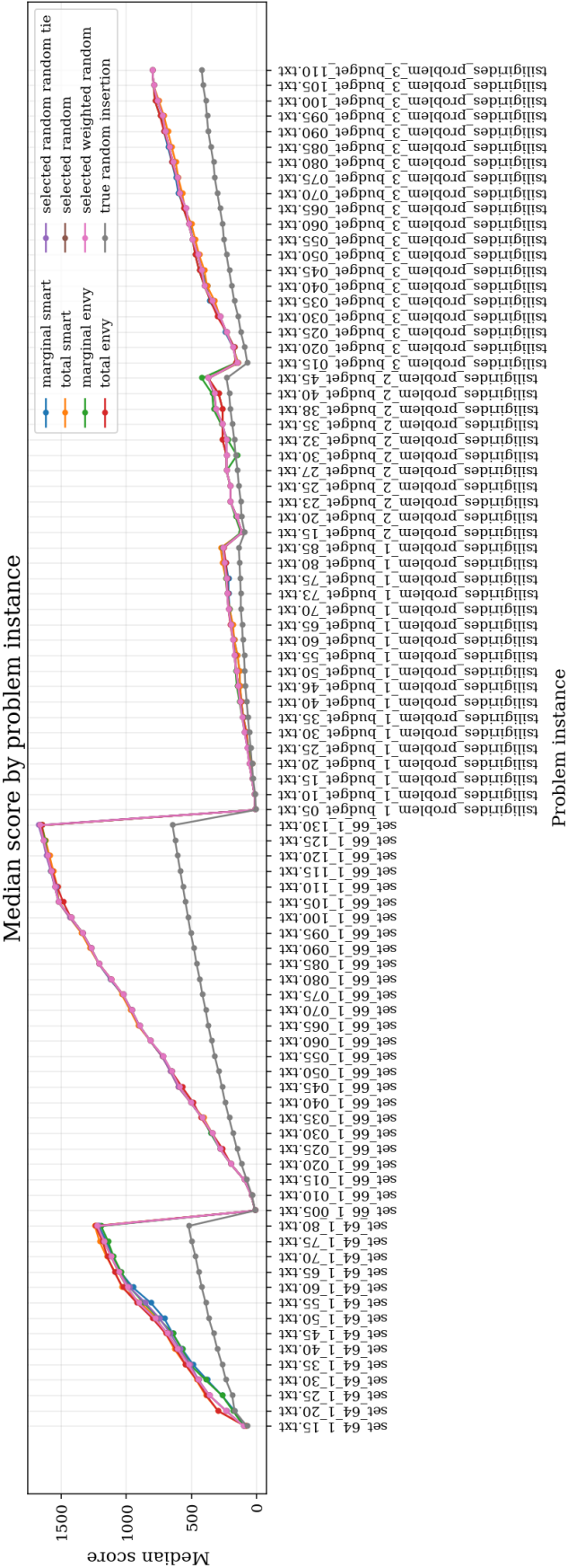
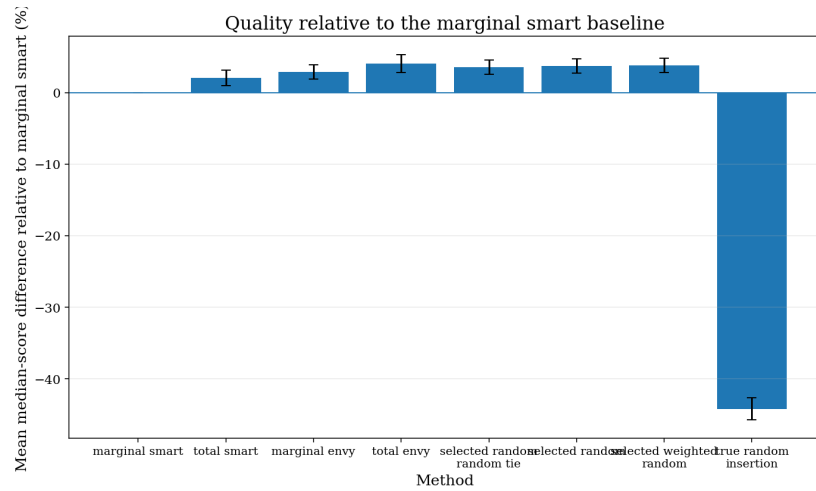
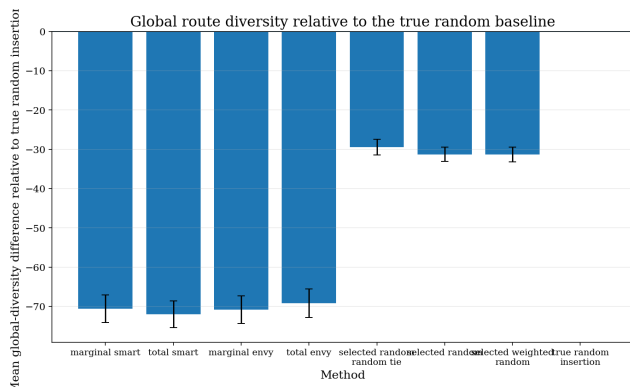


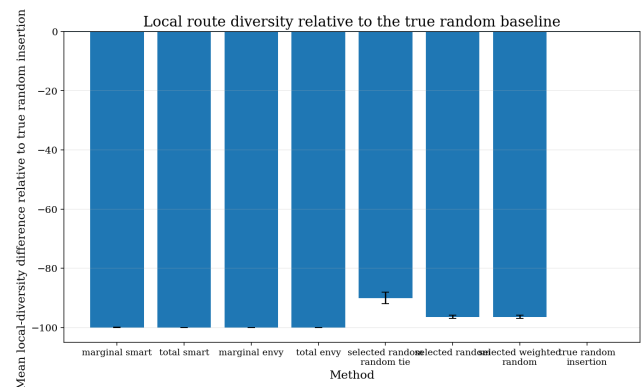
Figure 8. Median collected score for the selected deterministic, semi-randomized, and pure-random construction methods.



(a) Quality relative to marginal-smart.



(b) Global diversity relative to pure random.



(c) Local diversity relative to pure random.

Figure 9. Quality and structural diversity of deterministic, semi-randomized, and pure-random methods. References differ by panel and are stated in the panel labels. Error bars are standard errors across benchmark instances.

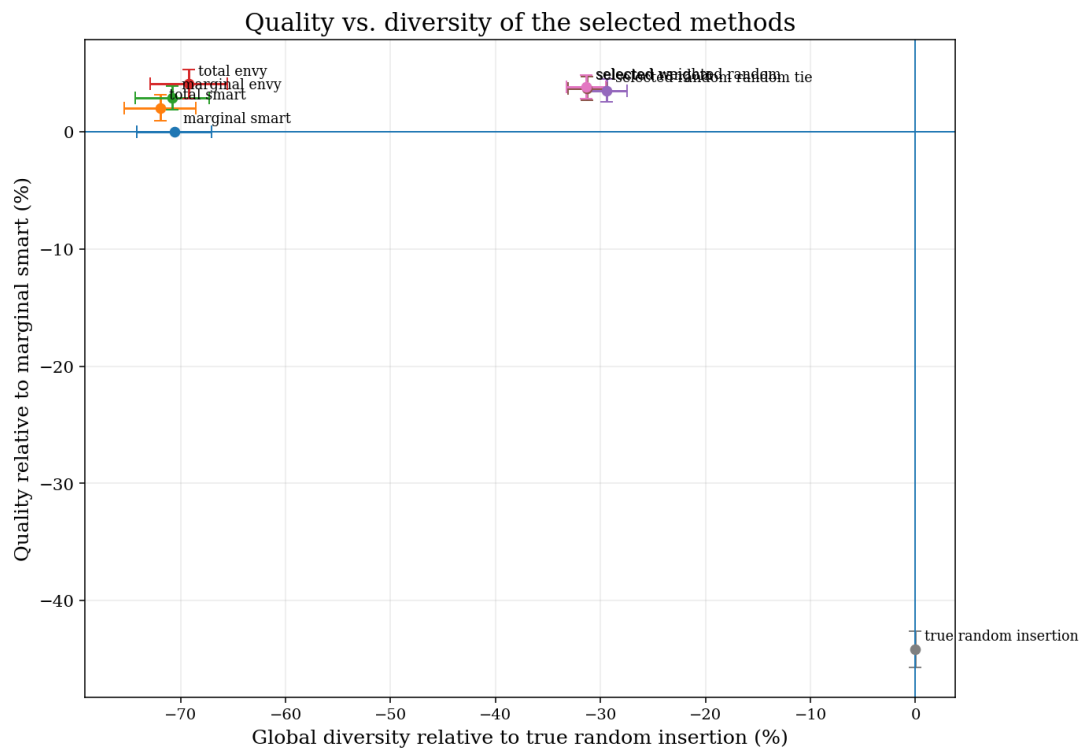


Figure 10. Quality–diversity trade-off. The vertical axis is mean median-score difference relative to marginal-smart; the horizontal axis is mean global-diversity difference relative to pure random.

5.7 Representative Route Structures

The aggregate results above are complemented by one unified visual comparison. It contains the same eight methods used in Section 5.6: marginal-smart, total-smart, marginal-envy, total-envy, selected-random, selected-weighted-random, selected-random-random-tie, and random-feasible-insertion. Separate grids for the elementary criteria are omitted because they would repeat a subset of this comparison without adding evidence.

For each method–instance pair, exactly 10,000 routes were constructed. Exact duplicate route vectors were merged and their observed frequencies retained. From the unique routes, at most six representatives were selected using the directed-arc Jaccard distance in Equation (7). The selection prioritizes the largest minimum pairwise distance and uses the sum of pairwise distances as a secondary objective. The implementation starts from a farthest pair, completes the set by max–min greedy selection, and applies one-swap improvements. This is a diversity-oriented extraction heuristic, not a proof that the globally most diverse six-route subset was found.

Figures 11–13 show three distinct benchmark families chosen from the earlier diversity analysis. Light circles are all instance vertices; colored vertices and edges form the route; the square is the start; and the triangle is the finish. Each occupied panel reports route-level statistics generated by the plotting script. A blank cell means that fewer than six unique route structures occurred in the 10,000 executions; it does not indicate missing data. These figures are qualitative structural checks and are not used to rank methods.

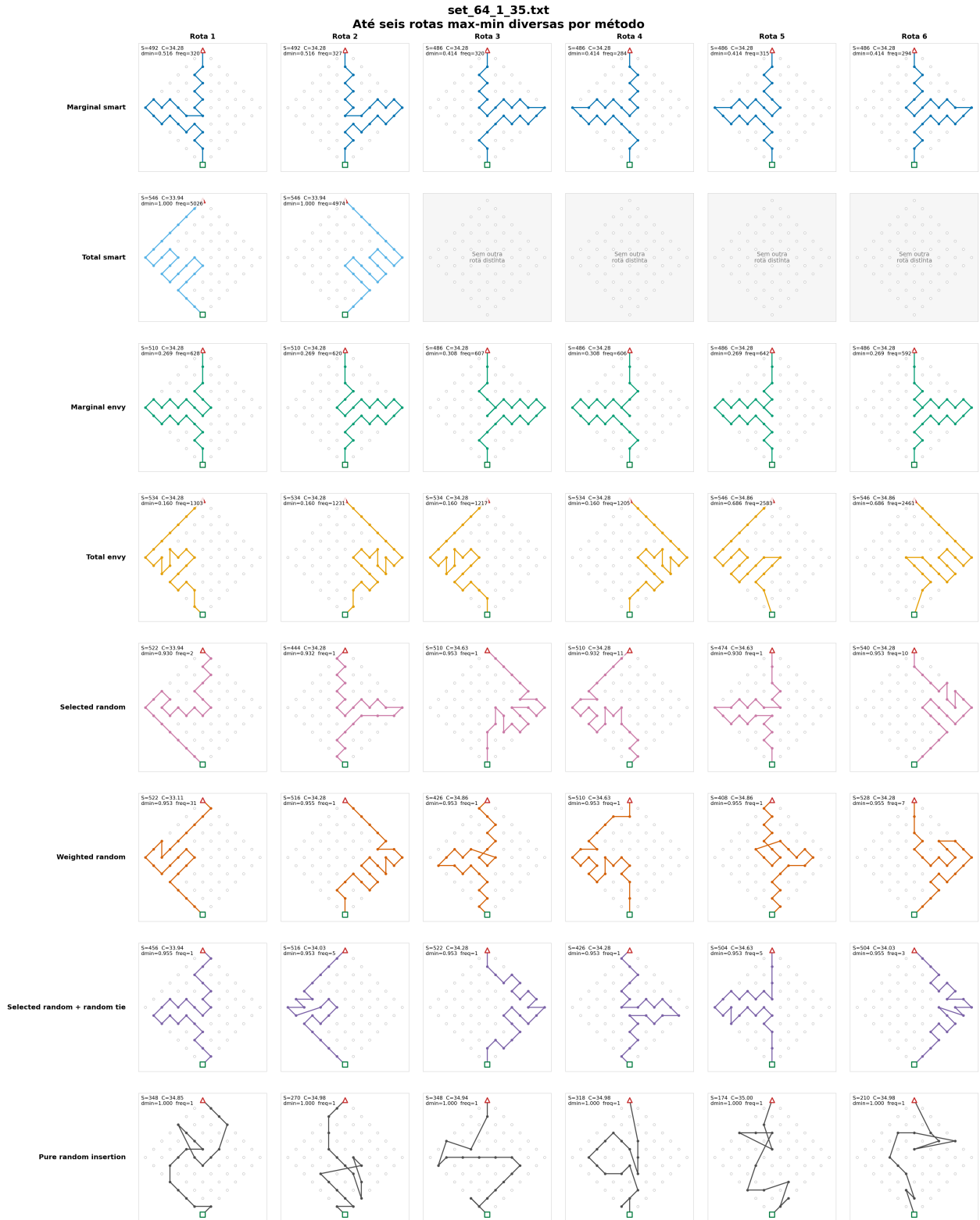


Figure 11. Max-min diverse route representatives for set_64_1_35. Rows are the eight final-comparison methods and columns are up to six representatives selected from 10,000 executions.

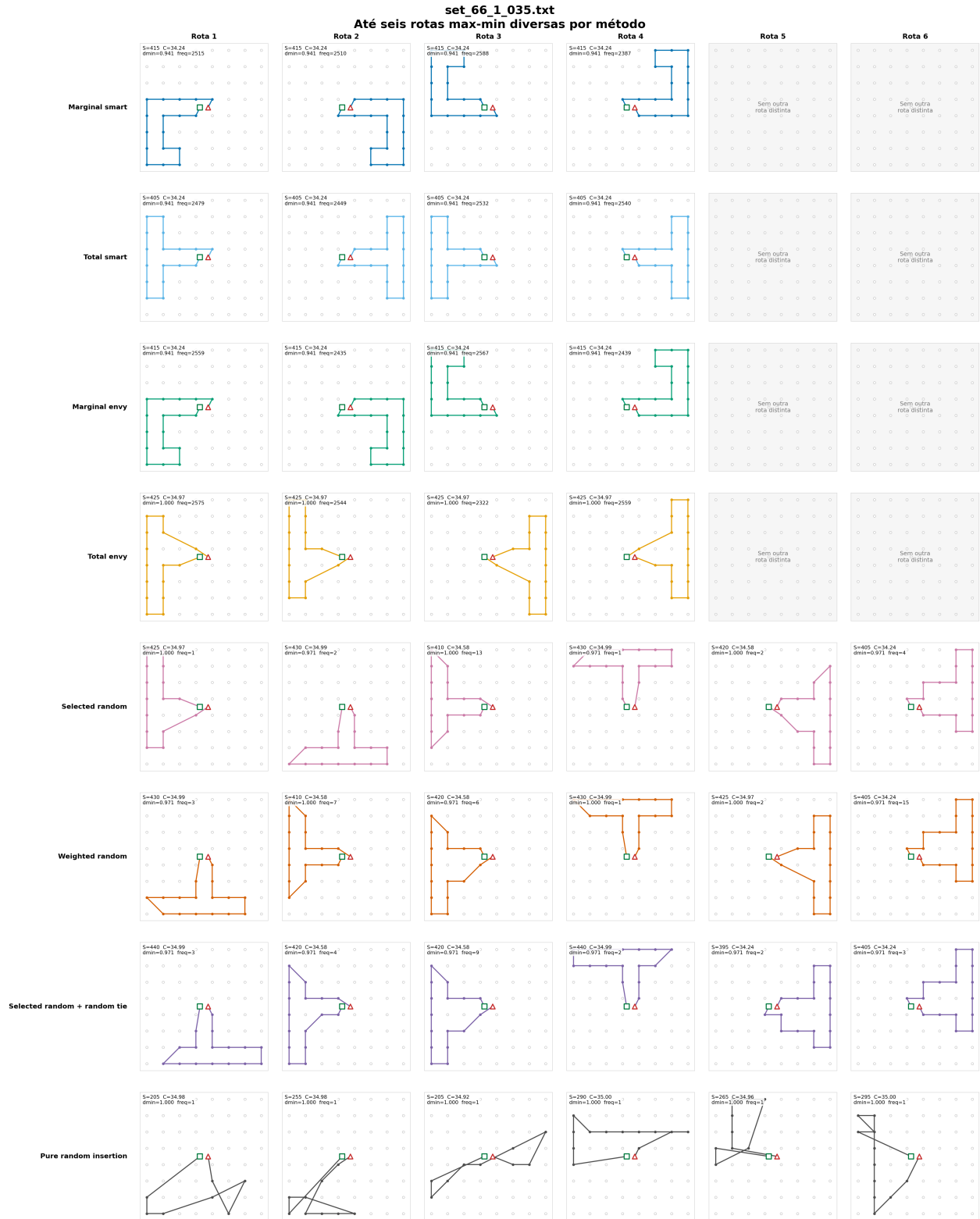


Figure 12. Max-min diverse route representatives for set_66_1_035. Blank cells mean that fewer than six unique route structures occurred in 10,000 executions.

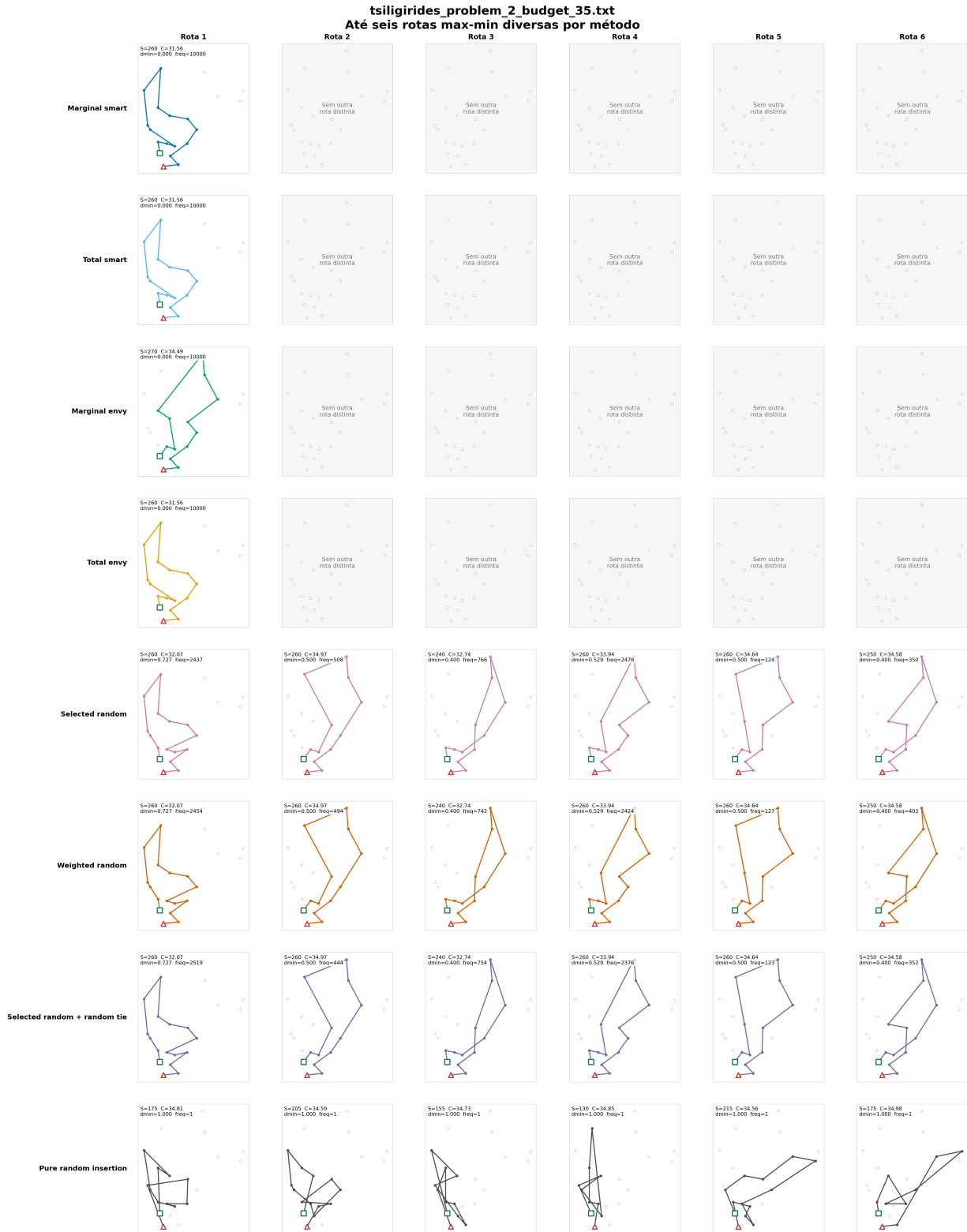


Figure 13. Max-min diverse route representatives for `tsiligirides_problem_2_budget_35`. The layout and method order are the same as in Figures 11 and 12.

6 Conclusion

This study compared constructive insertion rules for the OP within one common framework. Separating candidate generation from selection made the comparison linear and exposed an important distinction: marginal and total formulations of nonlinear score–cost criteria are not equivalent.

Total-envy achieved the largest mean deterministic improvement over marginal-smart, approximately 4.07%. Guided semi-randomized methods achieved 3.55–3.80% while more than doubling global structural diversity relative to the strongest deterministic criteria. Pure-random insertion, by contrast, lost approximately 44.17% in mean median score. The evidence therefore favors guided randomization rather than randomness alone.

Authors' Contributions

João Pereira conceived the study, implemented the algorithms, executed the experiments, analyzed the results, and prepared the manuscript.

Competing interests

The author declares no conflicts of interest.

Further relevant information

This computational study did not involve human participants, personal data, animals, or interventions requiring ethics-committee approval.

References

- Chao, I.-M., Golden, B. L., and Wasil, E. A. (1996). A fast and effective heuristic for the orienteering problem. *European Journal of Operational Research*, 88(3):475–489. DOI: 10.1016/0377-2217(95)00035-6.
- Dang, D.-C., Guibadj, R. N., and Moukrim, A. (2013). An effective PSO-inspired algorithm for the team orienteering problem. *European Journal of Operational Research*, 229(2):332–344. DOI: 10.1016/j.ejor.2013.02.049.
- Golden, B. L., Levy, L., and Vohra, R. (1987). The orienteering problem. *Naval Research Logistics*, 34(3):307–318. DOI: 10.1002/1520-6750(198706)34:3<307::AID-NAV3220340302>3.0.CO;2-D.
- Gunawan, A., Lau, H. C., and Vansteenwegen, P. (2016). Orienteering problem: A survey of recent variants, solution approaches and applications. *European Journal of Operational Research*, 255(2):315–332. DOI: 10.1016/j.ejor.2016.04.059.
- Kantor, M. G. and Rosenwein, M. B. (1992). The orienteering problem with time windows. *Journal of the Operational Research Society*, 43(6):629–635.
- Kobeaga, G., Merino, M., and Lozano, J. A. (2018). An efficient evolutionary algorithm for the orienteering problem. *Computers & Operations Research*, 90:42–59. DOI: 10.1016/j.cor.2017.09.003.
- Kobeaga, G., Rojas-Delgado, J., Merino, M., and Lozano, J. A. (2024). A revisited branch-and-cut algorithm for large-scale orienteering problems. *European Journal of Operational Research*, 313(1):44–68. DOI: 10.1016/j.ejor.2023.07.034.
- KU Leuven, Mechatronic Systems, Robotics and Biomechanics (2026). Orienteering problem: Test instances. <https://www.mech.kuleuven.be/en/mim/op>. Accessed 20 September 2026.
- Tsiligirides, T. (1984). Heuristic methods applied to orienteering. *Journal of the Operational Research Society*, 35(9):797–809. DOI: 10.1057/jors.1984.162.
- Vansteenwegen, P., Souffriau, W., and Van Oudheusden, D. (2011). The orienteering problem: A survey. *European Journal of Operational Research*, 209(1):1–10. DOI: 10.1016/j.ejor.2010.03.045.
- Vidal, T., Crainic, T. G., Gendreau, M., and Prins, C. (2013). A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows. *Computers & Operations Research*, 40(1):475–489. DOI: 10.1016/j.cor.2012.07.018.